Estratégia Concursos

Aula 17

DataPrev (Analista de TI - Desenvolvimento de Software) Passo Estratégico de Conhecimentos Específicos - 2024 (Pós-Edital)

Autor:

09 de Setembro de 2024

11225354498 - Maria Vitória Gonçalves Vieira

Análise e Projeto Orientado a Objetos

Sumário

Conteúdo 2
Glossário de termos 3
Roteiro de revisão 5
- Introdução a Orientação a Objetos 5
- Objetos 9
- Classes 12
- Atributos 15
- Métodos 16
- Mensagens 18
- Abstração 19
- Interface 21
- Encapsulamento 22
- Herança (Generalização/Especialização) 24
- Polimorfismo 29
- Análise e Projeto OO 34

Aposta estratégica 40
Questões Estratégicas 41
Questionário de revisão e aperfeiçoamento 43
- Perguntas 44
- Perguntas e Respostas 45

Lista de Questões Estratégicas 47

Conteúdo

Orientação a Objetos. Objetos. Classes. Atributos. Métodos. Interface. Abstração. Mensagens. Encapsulamento. Herança. Polimorfismo. Análise e Projeto OO.

11225354498 - Maria Vitória Gonçalves Vieira

Análise Estatística

Inicialmente, convém destacar o percentual de incidência do assunto, dentro da disciplina Engenharia de Software em concursos/cargos similares. Quanto maior o percentual de cobrança de um dado assunto, maior sua importância.

Obs.: *um mesmo assunto pode ser classificado em mais de um tópico devido à multidisciplinaridade de conteúdo*.

Assunto	Relevância na disciplina em concursos similares
UML	**11.0 %**
Processos de Software - Desenvolvimento Ágil	**8.4 %**
Engenharia de Requisitos	**8.2 %**
Teste de Software	**6.7 %**
Métricas de Software	**5.1 %**
Desenvolvimento de Software	**4.6 %**
Inteligencia Artificial	**4.6 %**
Processos de Software	**4.6 %**
Orientação a Objetos	**4.5 %**
Gestão de Projetos em Engenharia de Software	**3.6 %**
Qualidade de Software	**3.4 %**
Metodologia de desenvolvimento de software	**2.5 %**
Gerência de Configuração	**1.9 %**
Geoprocessamento em Engenharia de Software	**1.6 %**
Prototipação	**1.2 %**

11225354498 - Maria Vitória Gonçalves Vieira

Conceitos Básicos em Engenharia de Software	**0.7 %**
Análise Estruturada	**0.6 %**
Ferramentas CASE	**0.4 %**
Ferramentas de Desenvolvimento de Software	**0.4 %**
Software livre	**0.3 %**
Manutenção de Software	**0.1 %**
Análise Essencial	**0.1 %**
Web 2.0	**0.1 %**
Portal Web	**0.1 %**
Refatoração	**0.1 %**
Engenharia da Informação	**0.1 %**

GLOSSÁRIO DE TERMOS

Faremos uma lista de termos que são relevantes ao entendimento do assunto desta aula. Caso tenha alguma dúvida durante a leitura, esta seção pode lhe ajudar a esclarecer.

Orientação a objetos: Paradigma de programação que usa objetos e suas interações para projetar aplicações e programas de software.

Objeto: Uma instância de uma classe que possui um estado e comportamento definidos.

Identidade: Característica única de um objeto que o distingue de outros objetos.

Estado: Conjunto de valores associados a um objeto em um ponto específico no tempo.

11225354498 - Maria Vitória Gonçalves Vieira

Comportamento: Conjunto de métodos em uma classe de objeto que definem como o objeto atua e reage.

Abstração: Simplificação de sistemas complexos ao modelar apenas características relevantes para o contexto.

Encapsulamento: Prática de esconder os detalhes da implementação de um objeto e exibir apenas funcionalidades ao usuário.

Herança: Propriedade que permite que classes compartilhem atributos e métodos, formando uma hierarquia de classes.

Polimorfismo: Capacidade de um objeto ser interpretado como mais de um tipo.

Herança Múltipla: Característica que permite a uma classe herdar comportamentos e atributos de mais de uma superclasse.

Instância: Um objeto específico criado a partir de uma classe.

Atributo: Variáveis que definem as propriedades ou estado de um objeto.

Método: Funções definidas dentro de uma classe, que representam o comportamento dos objetos.

Classe Abstrata: Classe que não pode ser instanciada e é geralmente usada como uma classe base para outras classes.

Atributo de Classe: Variável que é compartilhada por todas as instâncias de uma classe.

Método Construtor: Método especial de uma classe usado para inicializar novos objetos dessa classe.

Mensagem: Forma de comunicação entre objetos onde um objeto solicita que outro execute um de seus métodos.

Early Binding (Ligação Precoce): Situação em que o método a ser executado é determinado em tempo de compilação.

Late Binding (Ligação Tardia): Situação em que o método a ser executado é determinado em tempo de execução.

11225354498 - Maria Vitória Gonçalves Vieira

Interface: Contrato dentro de um sistema orientado a objetos que especifica quais métodos uma classe deve implementar.

Modificador de Acesso: Palavras-chave que definem a visibilidade dos atributos, métodos e outras partes de uma classe ou estrutura.

Overriding (Sobrescrita): Ocorre quando uma subclasse fornece uma implementação diferente de um método já existente na superclasse.

Overloading (Sobrecarga): Ocorre quando dois ou mais métodos na mesma classe têm o mesmo nome, mas parâmetros diferentes.

Classe de Fronteira: Classe que controla a interação entre o sistema e o ambiente externo, geralmente através de interfaces de usuário.

Classe de Controle: Classe responsável por gerenciar o fluxo de controle entre classes de fronteira e classes de entidade.

Classe de Entidade: Classe que contém informações que devem ser salvas no sistema, geralmente correspondendo a um conceito do mundo real.

Roteiro de revisão

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

Introdução a Orientação a Objetos

Orientação a Objetos é um paradigma (modo de pensar) baseado no conceito de "objetos". Objetos são entidades que possuem **identidade** (algo que torna o objeto único), **estado** (valor corrente de seus atributos e propriedades) e **comportamento** (conjunto de funções ou métodos).

O Paradigma OO se contrapõe ao enfoque tradicional, que compreende o sistema como um conjunto de programas que executam processos sobre dados. No enfoque orientado a objetos, o sistema é uma coletânea de objetos que interagem entre si,

11225354498 - Maria Vitória Gonçalves Vieira

com características próprias representadas por dados (atributos) e operações (métodos)

Veja as principais vantagens do Paradigma OO:

VANTAGENS DO PARADIGMA ORIENTADO A OBJETOS
Produção de software natural. Os programas naturais são mais inteligíveis. Em vez de programar em termos de regiões de memória, o profissional pode programar usando a terminologia de seu problema em particular.
Programas orientados a objetos, bem projetados e cuidadosamente escritos são confiáveis.
Pode-se reutilizar prontamente classes orientadas a objetos bem-feitas. Assim como os módulos, os objetos podem ser reutilizados em muitos programas diferentes.
Um código orientado a objetos bem projetado é manutenível. Para corrigir um erro, o programador simplesmente corrige o problema em um lugar. Como uma mudança na implementação é transparente, todos os outros objetos se beneficiarão automaticamente do aprimoramento.
O software não é estático. Ele deve crescer e mudar com o passar do tempo, para permanecer útil. A programação orientada a objetos apresenta ao programador vários recursos para estender código. Esses recursos incluem herança, polimorfismo, sobreposição e uma variedade de padrões de projeto.
O ciclo de vida do projeto de software moderno é frequentemente medido em semanas. A programação orientada a objetos ajuda nesses rápidos ciclos de desenvolvimento. Ela diminui o tempo do ciclo de desenvolvimento, fornecendo software confiável, reutilizável e facilmente extensível.

11225354498 - Maria Vitória Gonçalves Vieira

História e Evolução da POO

A Programação Orientada a Objetos (POO) não surgiu de repente, mas foi o resultado de um processo evolutivo de décadas no campo da ciência da computação. A seguir, um breve resumo da história e origem da POO:

Simula (1960s)

A POO tem suas raízes na linguagem de programação Simula, desenvolvida na década de 1960 na Noruega por Ole-Johan Dahl e Kristen Nygaard. Simula foi a primeira linguagem de programação a introduzir os conceitos de classes, objetos, herança e subclasses, que são fundamentais para a POO.

Smalltalk (1970s)

O desenvolvimento da POO continuou na década de 1970 com a linguagem de programação Smalltalk, criada no Xerox Paloa Alto Research Center (PARC) por Alan Kay, Dan Ingalls e Adele Goldberg. Smalltalk foi a primeira linguagem a usar a terminologia "orientado a objetos" e a introduzir o conceito de encapsulamento. Smalltalk também é conhecida por seu ambiente de desenvolvimento gráfico, um dos primeiros ambientes de desenvolvimento integrado (IDEs).

C++ (1980s)

Na década de 1980, Bjarne Stroustrup desenvolveu a linguagem de programação C++, uma extensão da linguagem C que adicionou suporte para a programação orientada a objetos, incluindo classes, herança e polimorfismo. C++ tornou-se uma das linguagens de programação mais populares e influenciou o desenvolvimento de muitas outras linguagens de programação orientadas a objetos.

Java, Python e outras linguagens modernas (1990s-presente)

A linguagem de programação Java, desenvolvida pela Sun Microsystems na década de 1990, adotou uma abordagem puramente orientada a objetos, onde todos os programas são feitos de classes e objetos. Python, outra linguagem popular, também suporta a programação orientada a objetos, embora também suporte outros estilos de programação.

As ideias por trás da POO continuaram a evoluir e influenciar o desenvolvimento de novas linguagens e frameworks, como C#, Ruby, Swift e muitos outros.

11225354498 - Maria Vitória Gonçalves Vieira

Essa evolução continua até hoje, com novos paradigmas, como a programação funcional e a programação reativa, que oferecem novas formas de pensar sobre como o código deve ser estruturado e organizado. Mas, mesmo assim, a programação orientada a objetos continua sendo uma das técnicas fundamentais de qualquer desenvolvedor de software.

Paradigma OO versus Paradigma Estruturado

A programação estruturada e a programação orientada a objetos (POO) são dois paradigmas de programação importantes com diferenças significativas na maneira como os programas são concebidos e estruturados.

Programação Estruturada:

A programação estruturada enfatiza a **sequência lógica de instruções**. Ela foi projetada para tornar o controle de fluxo de programas mais previsível e fácil de entender.

Características da programação estruturada incluem:

- **Top-Down**: Os programas são geralmente desenvolvidos usando uma abordagem top-down, começando com uma visão geral do sistema e depois dividindo-o em partes menores que são mais gerenciáveis.
- **Blocos de Construção**: As principais estruturas de controle são sequência, seleção (if/else) e repetição (for, while). Essas estruturas permitem que o programa execute um bloco de código baseado em condições específicas.
- **Manipulação de Dados**: Os dados são geralmente manipulados diretamente usando estruturas de dados, como arrays e registros.
- **Procedimentos e Funções**: Em geral, a lógica do programa é dividida em procedimentos e funções que operam nos dados.
- **Limitações**: A programação estruturada pode se tornar complexa e difícil de gerenciar à medida que o tamanho do programa cresce. Além disso, a reutilização de código é limitada.

Programação Orientada a Objetos (POO):

A programação orientada a objetos se baseia no conceito de "objetos", que são instâncias de "classes". Cada objeto contém dados na forma de campos (também conhecidos como atributos ou propriedades) e código na forma de procedimentos (conhecidos como métodos).

Características da POO incluem:

11225354498 - Maria Vitória Gonçalves Vieira

- Abstração: Os objetos são abstrações do mundo real que encapsulam estado (dados) e comportamento (métodos).
- Encapsulamento: O encapsulamento é a prática de esconder os detalhes internos de como um objeto funciona e expondo apenas o que é necessário.
- Herança: A herança permite que as classes compartilhem comportamento e atributos. Uma classe "filha" pode herdar características de uma classe "pai".
- Polimorfismo: O polimorfismo permite que uma única interface seja usada para representar diferentes comportamentos de objetos.
- Benefícios: POO torna os sistemas mais fáceis de entender, manter e modificar, permitindo a reutilização de código e a redução da complexidade.

Objetos

Em programação orientada a objetos (POO), um objeto é uma entidade que combina estado e comportamento. Essencialmente, é uma instância de uma classe.

Identidade:

Identidade é uma propriedade do objeto que o distingue de todos os outros objetos do sistema. Algumas linguagens implementam isso como um *hashcode* outras como um "object ID". Em geral, o programador não precisa se preocupar com esse detalhe, pois o próprio ambiente da linguagem de programação trata isso de maneira transparente. O importante é saber que cada objeto tem uma referência ou identidade diferente dos demais, o que permite lhe identificar univocamente dentro do sistema.

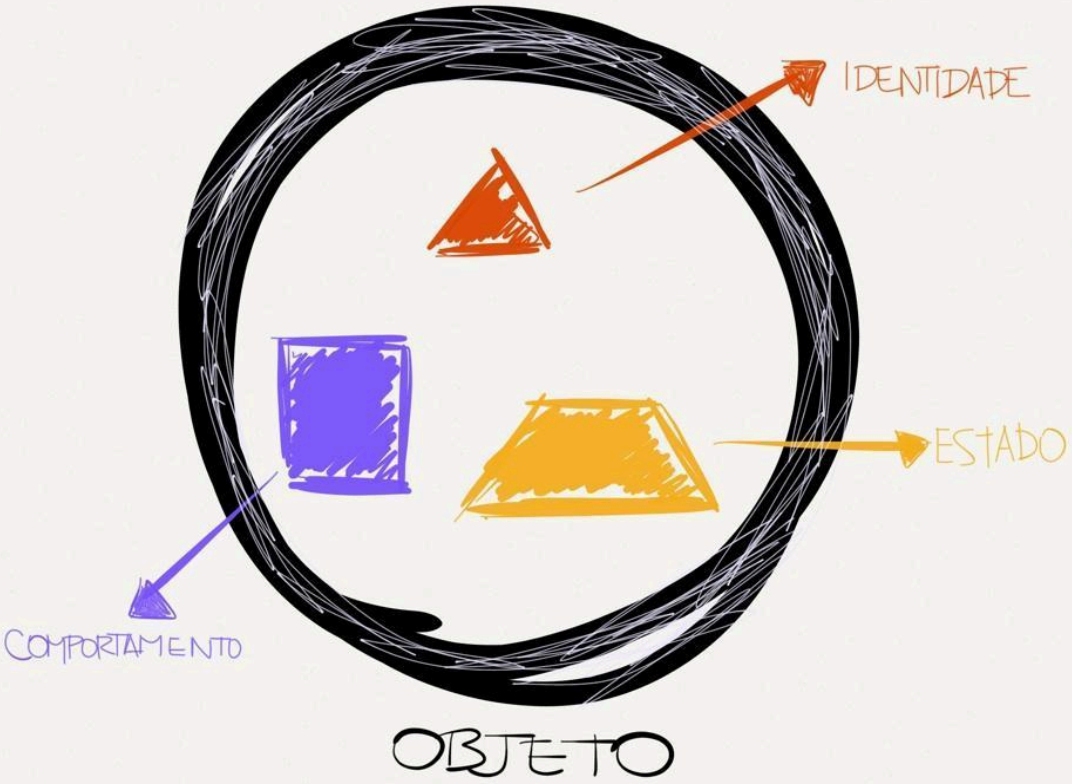
Estado:

O estado de um objeto é representado pelos valores correntes de seus atributos (também chamados de propriedades ou campos). Esses atributos são geralmente variáveis que contêm dados. Por exemplo, se considerarmos uma classe "Carro", os atributos do objeto poderiam incluir coisas como cor, marca, modelo e velocidade atual.

Comportamento:

O comportamento de um objeto é definido pelos métodos (também conhecidos como funções ou procedimentos) associados a esse objeto. Usando novamente o exemplo da classe "Carro", os métodos poderiam incluir coisas como acelerar, frear, ligar e desligar.

11225354498 - Maria Vitória Gonçalves Vieira



- Encapsulamento:

  Os objetos encapsulam tanto o estado quanto o comportamento, o que significa que eles escondem os detalhes de implementação e expõem apenas aquilo que é necessário. Isso é uma maneira de garantir a integridade dos dados e é um aspecto fundamental da programação orientada a objetos.

  Instâncias de uma Classe:

  É importante notar que um objeto é uma instância de uma classe. A classe define quais atributos e métodos estão associados a um objeto, mas não determina o valor desses atributos para um objeto específico. Por exemplo, a classe "Carro" define que cada carro tem uma cor, mas não especifica qual é essa cor - essa informação é específica para cada objeto (instância) da classe.

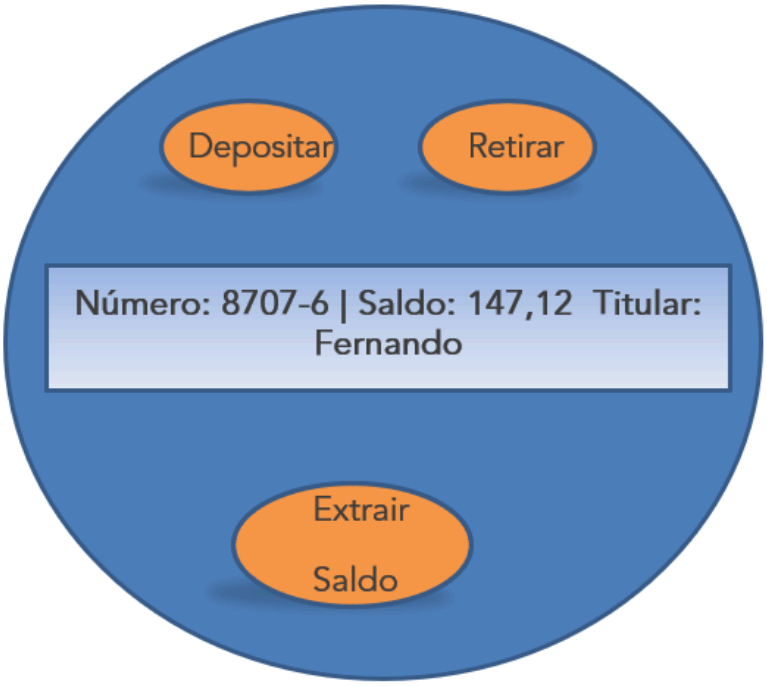
  Quando um objeto é criado (ou instanciado) a partir de uma classe, o processo é chamado de instanciação. Cada objeto instanciado a partir da mesma classe tem seu próprio estado, mas compartilha o comportamento definido pela classe.

  Para usar uma metáfora, você pode pensar em uma classe como um plano de projeto e o objeto como a casa que é construída com base nesse plano. O plano de projeto (a classe) define a estrutura e o layout (os atributos e métodos), mas não determina coisas

11225354498 - Maria Vitória Gonçalves Vieira

específicas como a cor das paredes ou o tipo de piso (o estado do objeto). Cada casa construída a partir do mesmo plano é diferente (um objeto separado), mas todas compartilham a mesma estrutura básica e layout (a classe).

Veja um exemplo de um objeto Conta Bancária:



Este objeto possui três atributos (número, saldo e titular) e três métodos (depositar, retirar e extrair saldo).

Classes

Uma classe é uma estrutura que define os atributos (ou propriedades) e os métodos (ou comportamentos) que são comuns a todos os objetos daquela classe. Uma classe serve como um "molde" ou um plano para criar (ou instanciar) objetos.

Por exemplo, podemos definir a classe Conta Bancária em Python da seguinte maneira:

```python
class Conta:
    # construtor da classe
    def __init__(self, numero, titular, saldo=0):
        self.numero = numero
        self.titular = titular
        self.saldo = saldo

    # método para debitar da conta
    def debitar(self, valor):
        if valor > self.saldo:
            print("Saldo insuficiente")
            return
        self.saldo -= valor

    # método para creditar na conta
    def creditar(self, valor):
        self.saldo += valor

    # método para extrair o saldo da conta
    def extrair_saldo(self):
        return self.saldo
```

Atributos:

A classe "Conta" possui três atributos: numero, titular e saldo. Estes atributos representam o estado do objeto e podem ter valores diferentes para cada objeto. Por exemplo, o atributo numero poderia ser o número da conta, o titular poderia ser o nome da pessoa que possui a conta, e o saldo seria a quantidade de dinheiro na conta.

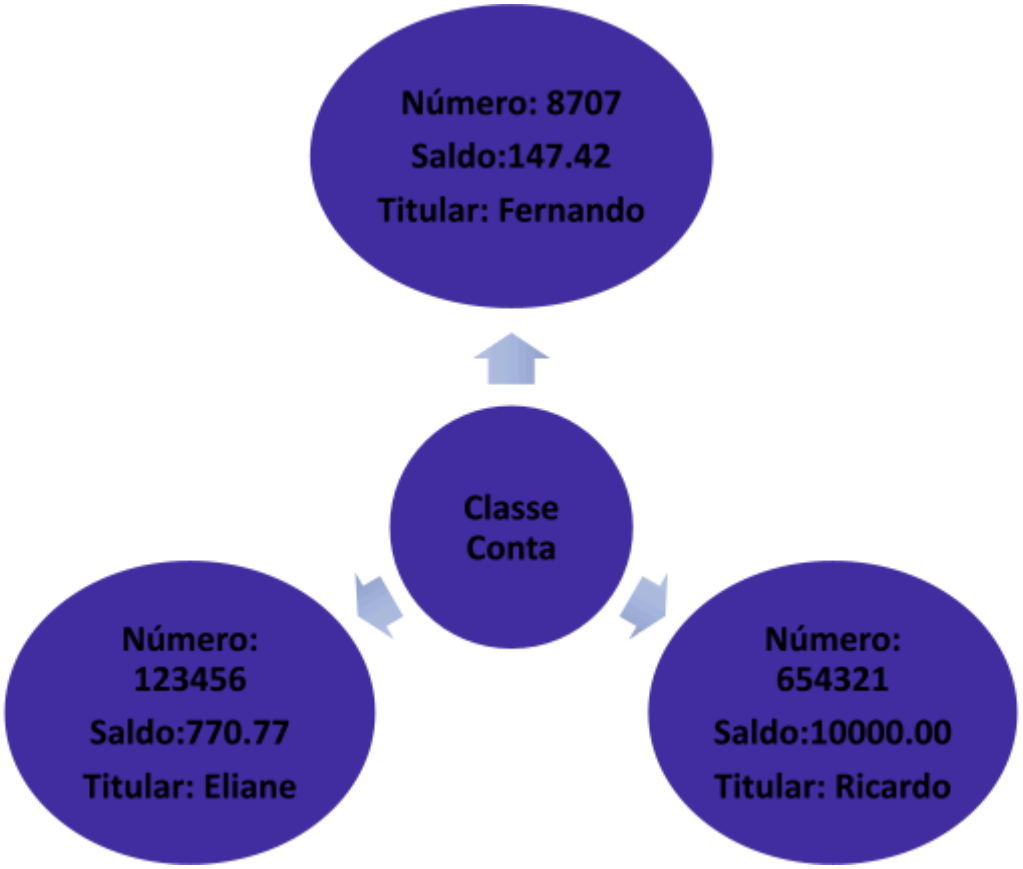
Métodos:

Os métodos da classe Conta definem o comportamento dos objetos da classe. No exemplo acima, os métodos são debitar, creditar e extrair_saldo.

11225354498 - Maria Vitória Gonçalves Vieira

- debitar: subtrai um valor do saldo da conta. Antes disso, verifica se o valor a ser debitado é maior do que o saldo atual da conta. Se for maior, uma mensagem de erro é exibida e a operação é cancelada.
- creditar: adiciona um valor ao saldo da conta.
- extrair_saldo: retorna o saldo atual da conta.

Veja que podemos criar diversos objetos "Conta Bancária" a partir desta mesma classe, porém com valores de atributos diferentes:



Em Python, para instanciar objetos poderíamos escrever um código como este:

```
conta1 = Conta(123, "João", 1000)
conta2 = Conta(456, "Maria", 2000)
```

Neste caso, conta1 e conta2 são dois objetos distintos, ambos instâncias da classe Conta. Cada um tem seus próprios atributos que são independentes um do outro.

Classe Abstrata

Uma classe abstrata, no contexto da programação orientada a objetos, é uma classe que não pode ser instanciada diretamente, ou seja, não é possível criar um objeto diretamente a partir de uma classe abstrata. Ela é geralmente usada como uma classe base para outras classes.

Classes abstratas são usadas para representar conceitos abstratos que não devem ser criados, mas apenas herdados.

Métodos abstratos funcionam como uma espécie de "contrato" que as subclasses devem seguir. Uma classe que herda de uma classe abstrata deve fornecer uma implementação para cada método abstrato da classe base.

Por exemplo, suponha que temos uma classe abstrata chamada Forma que representa uma forma geométrica:

```
from abc import ABC, abstractmethod

class Forma(ABC):

    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimetro(self):
        pass
```

Neste exemplo, Forma é uma classe abstrata que possui dois métodos abstratos: area e perimetro. Cada forma (como círculo, retângulo, triângulo, etc.) calcula a área e o perímetro de maneira diferente, por isso esses métodos são abstratos e devem ser implementados por qualquer classe que herde de Forma.

Veja o exemplo de uma classe Circulo que herda de Forma:

11225354498 - Maria Vitória Gonçalves Vieira

```python
import math

class Circulo(Forma):

    def __init__(self, raio):
        self.raio = raio

    def area(self):
        return math.pi * self.raio ** 2

    def perimetro(self):
        return 2 * math.pi * self.raio
```

Neste caso, a classe Circulo fornece a implementação para os métodos area e perimetro da classe base Forma.

Note que você não pode criar uma instância de Forma diretamente, porque é uma classe abstrata. No entanto, você pode criar uma instância de Circulo, que é uma subclasse concreta de Forma.

Atributos

O atributo consiste em uma informação de estado para o qual cada objeto de uma classe tem seu próprio valor. Há dois tipos: atributos de objetos e de classes.

Atributos de Objeto (Instância):

Os atributos de objeto, também conhecidos como atributos de instância, são associados a uma instância específica de uma classe. Cada instância (ou objeto) tem sua própria cópia desses atributos. Qualquer mudança feita a um atributo de instância não afetará os atributos de outras instâncias da mesma classe.

Por exemplo, considerando a classe "Conta" do exemplo anterior, os atributos numero, titular e saldo são atributos de instância porque cada instância da classe "Conta" (ou seja, cada objeto "Conta") terá seu próprio numero, titular e saldo:

11225354498 - Maria Vitória Gonçalves Vieira

Atributos de Classe:

Os atributos de classe, por outro lado, são associados à própria classe, e não a qualquer instância específica. Eles são compartilhados por todas as instâncias da classe. Portanto, qualquer mudança em um atributo de classe é refletida em todas as outras instâncias da classe.

Por exemplo, poderíamos ter um atributo de classe "tipo" na classe "Conta", que é o mesmo para todas as contas:

```
class Conta:
    tipo = "Conta Corrente"  # atributo de classe

    def __init__(self, numero, titular, saldo=0):
        self.numero = numero
        self.titular = titular
        self.saldo = saldo
```

Neste caso, "tipo" é um atributo de classe e será o mesmo para todas as instâncias de "Conta". Você pode acessar atributos de classe através da classe em si, Conta.tipo, ou através de uma instância, conta1.tipo.

Métodos

Um método é uma função que está associada a uma classe e é definida dentro dela. Métodos são usados para definir comportamentos dos objetos que são criados a partir da classe. Existem três tipos principais de métodos: métodos de instância, métodos construtor e métodos estáticos (de classe).

Método de Instância:

Métodos de instância são os mais comuns. Eles operam em uma instância específica da classe (ou seja, um objeto). Em Python, por exemplo, o primeiro argumento de um método de instância é sempre self, que é uma referência à instância atual da classe. Métodos de instância podem acessar e modificar o estado do objeto.

Por exemplo, na classe Conta, debitar, creditar e extrair_saldo são métodos de instância.

Método Construtor:

O método construtor é um tipo especial de método de instância que é chamado automaticamente quando um objeto é criado a partir de uma classe. Em Python, por exemplo, o método construtor é chamado __init__. É comumente usado para inicializar os atributos do objeto.

No exemplo da classe Conta, o método __init__ é usado para inicializar numero, titular e saldo quando uma nova Conta é criada:

```
class Conta:
    def __init__(self, numero, titular, saldo=0):  # método construtor
        self.numero = numero
        self.titular = titular
        self.saldo = saldo
```

Método de Classe:

Um método de classe opera na classe como um todo, não em uma instância específica da classe. Em Python, os métodos de classe são definidos usando o decorador @classmethod e seu primeiro argumento é sempre cls, que é uma referência à classe.

Os métodos de classe não podem acessar nenhum atributo de instância, mas podem acessar atributos de classe. Eles são comumente usados para criar métodos de fábrica, que são métodos alternativos para criar instâncias de uma classe.

Pense em um método de classe como um método "global", ou seja, trata-se de um método que realiza operações genéricas, isto é, não relativas a uma instância particular

A associação entre classe e método é chamada de ligação (Binding). Um método associado a uma classe é dito ligado (Bound) à classe. Existem dois tipos de ligações: Early Binding (Ligação Prematura) e Late Binding (Ligação Tardia):

TIPO DE ligação	DESCRIÇÃO
Early binding	Também conhecida como Ligação Estática, ocorre quando o método a ser invocado é definido em tempo de compilação.

11225354498 - Maria Vitória Gonçalves Vieira

Late binding	Também conhecida como Ligação Dinâmica, ocorre quando o método a ser invocado é definido em tempo de execução.

Mensagens

Qual é a utilidade de um objeto isolado? No contexto de OO, ele é geralmente muito pouco útil. Por meio da interação entre objetos é que se torna possível obter uma grande funcionalidade ou um comportamento mais complexo. Logo, assim como no mundo real, objetos estão sempre interagindo uns com os outros. Essa interação ocorre por meio de troca de mensagens.



Quando um objeto quer solicitar uma ação ou interagir com outro objeto, ele envia uma "mensagem" para o outro objeto. Envio de mensagem envolve essencialmente dois elementos principais:

- O objeto receptor: Este é o objeto para o qual a mensagem é enviada.
- A mensagem em si: A mensagem é geralmente uma chamada para um dos métodos do objeto receptor. Ela pode ter argumentos adicionais necessários para o método.

O envio de mensagens é a forma como os objetos interagem e colaboram para executar tarefas. Cada objeto é responsável por sua própria área de responsabilidade, e quando precisa de algo que está além de sua responsabilidade, ele enviará uma mensagem para o objeto apropriado pedindo para executar a tarefa.

Por exemplo, considere uma instância de uma classe Conta chamada conta1. Se quisermos depositar dinheiro na conta, podemos enviar a mensagem creditar para o objeto conta1 com o valor a ser depositado como um argumento:

```
conta1.creditar(500)
```

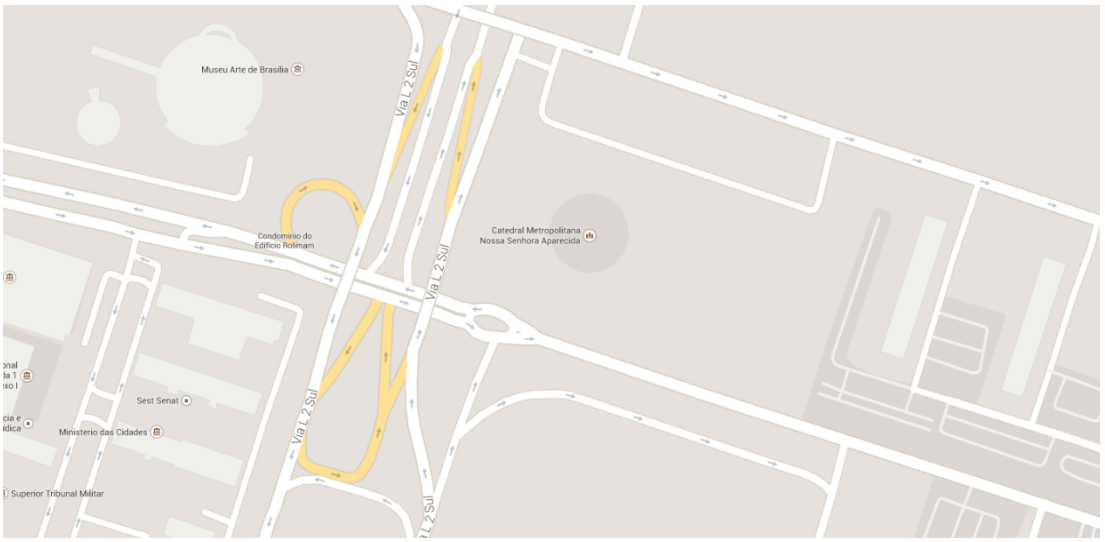
Neste caso, conta1 é o objeto receptor e creditar é a mensagem, com 500 sendo um argumento adicional necessário para o método. Ao receber a mensagem, conta1 executará o método creditar e atualizará seu saldo de acordo.

A ideia de enviar mensagens para objetos, em vez de simplesmente chamar funções, permite que você pense em termos de objetos que interagem uns com os outros, em vez de fluxos de controle, tornando o código mais fácil de entender, mais modular e mais fácil de manter.

Abstração

De modo simples e direto, abstração é a **subtração de detalhes**, isto é, **quanto mais abstrato, há menos detalhes; e quanto menos abstrato, há mais detalhes**. Observe as imagens abaixo: trata-se de uma visão aérea da Catedral de Brasília. Na primeira imagem, há muitos detalhes; a segunda imagem é uma abstração em que se subtraiu diversos detalhes que não são relevantes para o domínio do problema





A abstração permite que você reduza a complexidade focando nos aspectos essenciais de um objeto, sem se preocupar com os detalhes. Isso torna o código mais fácil de entender, mais limpo e mais eficiente. Também fornece um alto nível de modularidade em sua aplicação, pois cada objeto tem sua própria funcionalidade definida e é independente de outros objetos.

Interface

Uma interface é um **contrato que especifica um conjunto de métodos que uma classe deve implementar**. Ela define quais métodos uma classe deve ter, mas não fornece uma implementação para esses métodos. A classe que implementa a interface deve fornecer a implementação para todos os métodos especificados pela interface.

A interface é usada para garantir a consistência e o padrão entre várias classes. É uma maneira de garantir que diferentes classes adotem um conjunto comum de comportamentos, independentemente de suas implementações individuais.

É importante diferenciar classes abstratas de interfaces. Ambas possuem objetivos parecidos, mas têm algumas diferenças fundamentais, tais como:

características	Interfaces	Classe abstrata

Herança Múltipla	Suporta Herança Múltipla. Pode implementar diversas interfaces.	Não suporta Herança Múltipla. Não pode estender várias classes abstratas.
Implementação	Não pode conter qualquer método concreto, apenas abstratos.	Pode conter métodos concretos ou abstratos.
Constantes	Suporta somente constantes estáticas.	Suporta constantes estáticas e de instância.
Encapsulamento	Métodos e membros devem sempre ser públicos por padrão.	Métodos e membros podem ter qualquer visibilidade.
Membros de Dados	Não contêm atributos de instância, apenas constantes.	Pode conter atributos de instância.
Construtores	Não contém construtores.	Contém construtores.
Velocidade	Em geral, são mais lentas que classes abstratas.	Em geral, são mais rápidas que interfaces.

Pode-se dizer que uma interface é similar a uma classe abstrata. Aliás, podemos dizer que uma interface é praticamente uma classe abstrata pura, isto é, todos os seus métodos são abstratos. Uma classe concreta – ao implementar uma interface – deverá escrever o corpo de todos os métodos. Observe, portanto, que uma classe abstrata pode também implementar uma interface sem nenhum problema.

Dessa forma, conceitualmente, podemos dizer que uma classe abstrata especifica o que um objeto é; uma interface especifica o que um objeto pode fazer.

11225354498 - Maria Vitória Gonçalves Vieira

Encapsulamento

Encapsulamento é um dos três mecanismos principais da POO, juntamente com herança e polimorfismo. Ele descreve a ideia de agrupar dados (atributos) e os métodos que operam nesses dados em uma única unidade, que é chamada de objeto.

O encapsulamento também envolve **ocultar os detalhes internos de um objeto e expor apenas o que é necessário**. Em outras palavras, um objeto mantém seus estados (dados) privados e fornece métodos públicos (conhecidos como acessadores e mutadores, ou getters e setters) para acessar e modificar esses dados.

Principais benefícios do encapsulamento:

- **Controle de acesso**: O encapsulamento permite que os detalhes internos de um objeto sejam ocultados e protegidos de serem acessados ou modificados diretamente. Isso evita que o estado interno do objeto seja alterado de maneira inadequada ou acidental.
- **Flexibilidade e fácil manutenção**: Como os detalhes internos do objeto são ocultados, alterações no código interno de um objeto (por exemplo, como os dados são armazenados ou operações são realizadas) não afetarão os outros objetos ou partes do código que usam o objeto. Isso facilita a modificação e a manutenção do código.
- **Modularidade**: O encapsulamento facilita a divisão de um programa em várias partes independentes, ou módulos. Cada objeto é um módulo separado, com sua própria lógica e dados. Isso torna o código mais organizado e mais fácil de entender.

No contexto de encapsulamento, existe o conceito de Modificadores de Acessibilidade. Conhecidos também como visão de método ou visão de atributo, definem a visibilidade de um atributo, método ou classe. Em geral, utilizam-se especificadores de acesso para privar o acesso direto a atributos e obrigar o usuário a fazê-lo por meio de métodos públicos.

Veja os símbolos de modificadores de acessibilidade como apresentados na UML e em Java:

MODIFICADOR/ESPECIFICADOR	CLASSE	PACOTE	SUBCLASSE	TODOS

11225354498 - Maria Vitória Gonçalves Vieira

UML	PÚBLICO	+	x	x	x	x
	PROTEGIDO	#	x		x	
	PACOTE	~	x	x		
	PRIVADO	–	x			

MODIFICADOR/ESPECIFICADOR			CLASSE	PACOTE	SUBCLASSE	TODOS
JAVA	PÚBLICO	+	x	x	x	x
	PROTEGIDO	#	x	x	x	
	DEFAULT	~	x	x		
	PRIVADO	-	x			

Herança (Generalização/Especialização)

Herança, também conhecida como o mecanismo de Generalização ou Especialização (a depender do ponto de vista), é um conceito na programação orientada a objetos que permite **criar uma nova classe a partir de uma classe existente**. A nova classe é chamada de classe derivada ou classe filha, e a classe existente da qual ela é derivada é chamada de classe base ou classe pai.

A herança permite que a classe filha herde todos os atributos e métodos da classe pai, permitindo a reutilização do código. Além disso, a classe filha pode adicionar novos atributos e métodos ou pode sobrescrever os métodos herdados da classe pai.

Veja um exemplo simples de herança em Python:

```python
class Conta:
    def __init__(self, numero, titular):
        self.numero = numero
        self.titular = titular
        self.saldo = 0

    def creditar(self, valor):
        self.saldo += valor

    def debitar(self, valor):
        if valor > self.saldo:
            return False
        self.saldo -= valor
        return True

class ContaCorrente(Conta):
    def __init__(self, numero, titular, limite):
        super().__init__(numero, titular)
        self.limite = limite

    def debitar(self, valor):
        if valor > self.saldo + self.limite:
            return False
        self.saldo -= valor
        return True
```

Neste exemplo, ContaCorrente é uma classe filha de Conta. Ela herda os atributos numero, titular e saldo, e os métodos creditar e debitar da classe Conta. Além disso, ContaCorrente adiciona um novo atributo, limite, e sobrescreve o método debitar para levar em conta o limite de cheque especial ao debitar da conta.

Principais benefícios da herança:

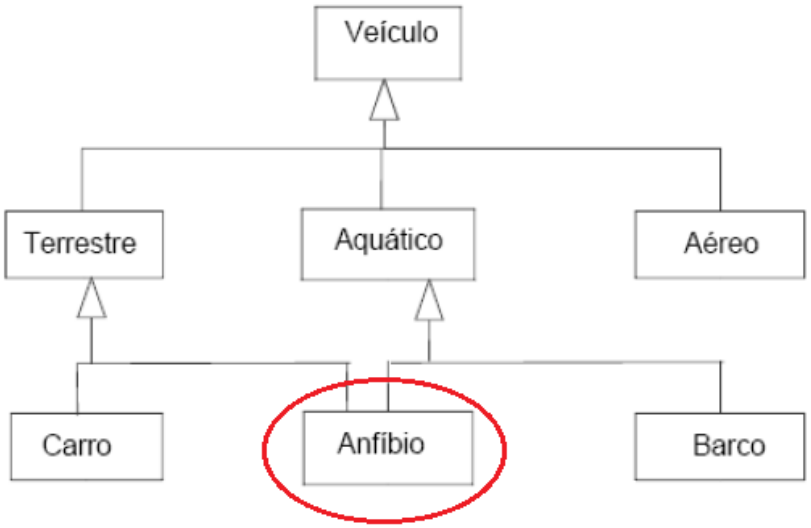
11225354498 - Maria Vitória Gonçalves Vieira

- **Reutilização de código**: A herança permite que as classes filhas reutilizem o código da classe pai. Isso reduz a quantidade de código que você precisa escrever e minimiza a redundância.
- **Extensibilidade**: Permite que você adicione mais funcionalidades a uma classe existente sem alterar a classe original. Isso é feito criando uma nova classe que herda da classe original e adicionando ou modificando métodos conforme necessário.
- **Fornecer estrutura hierárquica**: Fornece uma estrutura lógica e fácil de entender para o seu código. Isso pode tornar seu código mais fácil de ler e manter.
- **Modelagem de problemas do mundo real**: Na realidade, muitos conceitos são naturalmente hierárquicos (por exemplo, um gato é um tipo de animal). A herança permite que você modele essa hierarquia natural em seu código.

Herança Múltipla

A herança múltipla é um recurso de algumas linguagens de programação orientadas a objetos que permite que uma classe herde de duas ou mais classes. Isso significa que uma classe filha pode herdar métodos e variáveis de várias classes pai.

Veja um exemplo no diagrama abaixo:



Embora a herança múltipla possa parecer útil em alguns casos, muitas vezes é evitada porque pode levar a uma série de problemas complexos:

- **Problema do diamante**: Este é um problema clássico de herança múltipla onde uma classe herda de duas classes que têm uma classe base comum. Isso pode levar a

ambiguidades sobre qual método usar se a classe filha invocar um método que é definido em ambas as classes pai.

- Complexidade aumentada: A herança múltipla pode tornar o design do seu código muito mais complexo e difícil de entender, especialmente à medida que o número de classes pai aumenta.
- Acoplamento de código: Pode levar a um alto grau de acoplamento entre as classes, tornando o código mais difícil de modificar e manter.
- Sobrescrita de método: Se duas classes pai tiverem métodos com o mesmo nome, pode ser confuso determinar qual deles a classe filha está realmente herdando.

Por esses motivos, muitas linguagens modernas de programação orientadas a objetos, como Java e C#, evitam a herança múltipla e, em vez disso, usam interfaces ou outros artifícios. Algumas linguagens, como Python e C++, ainda permitem a herança múltipla, mas os desenvolvedores são incentivados a usá-la com cautela.

Restrições de Generalização

Quando se discute generalização, é útil definir algumas propriedades que podem ajudar a restringir e esclarecer o relacionamento entre as classes.

Generalização completa versus incompleta:

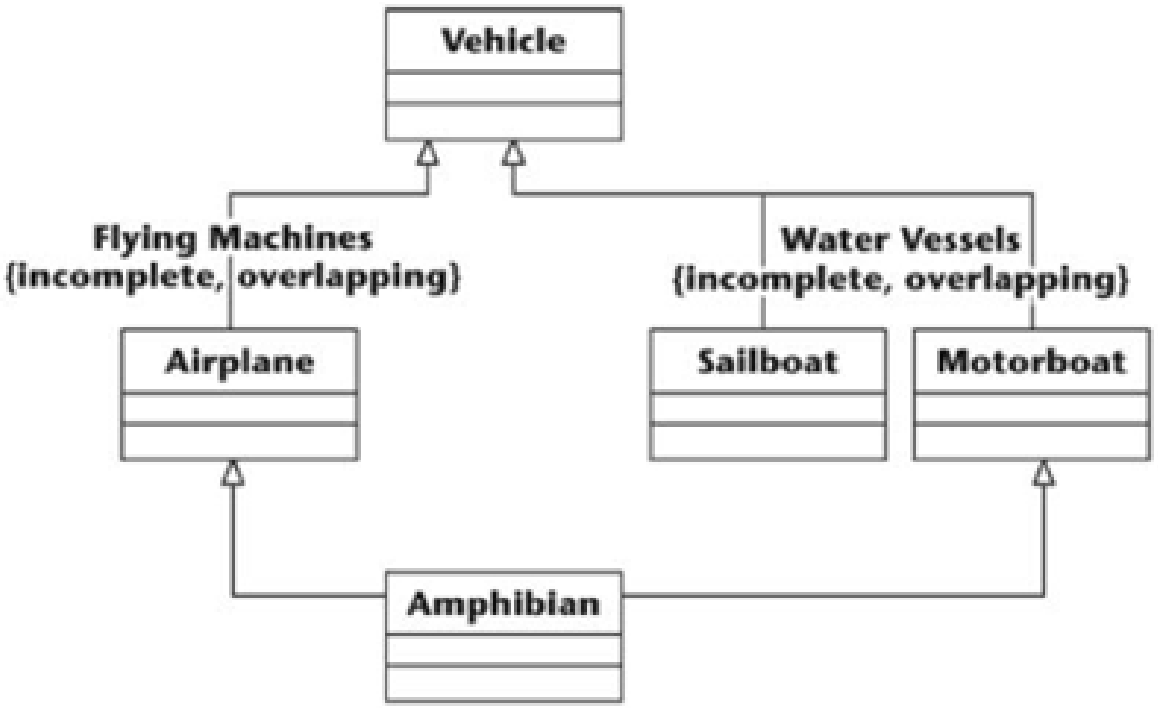
- Generalização completa: Isso ocorre quando todos os casos possíveis da superclasse são cobertos pelas subclasses. Ou seja, cada instância da superclasse corresponde a pelo menos uma das subclasses.
- Generalização incompleta: Isso ocorre quando nem todos os casos da superclasse são cobertos pelas subclasses. Existem algumas instâncias da superclasse que não correspondem a nenhuma das subclasses.

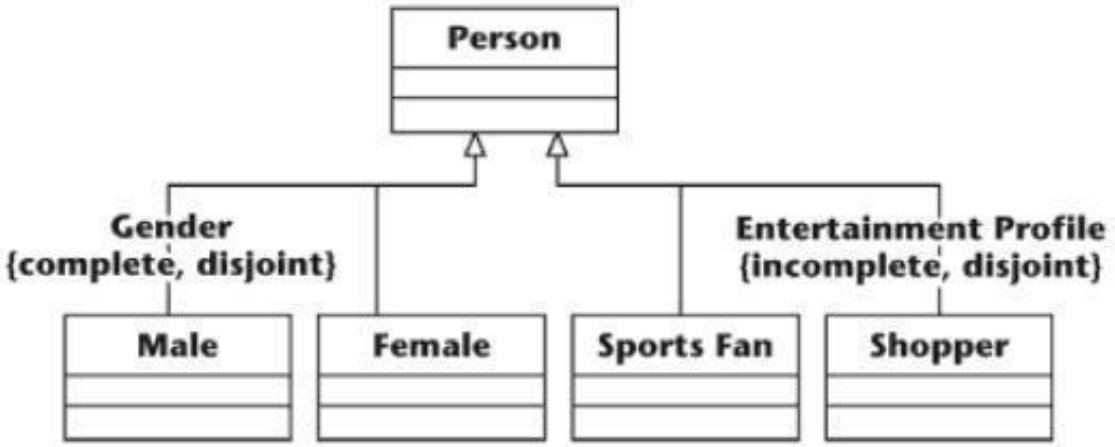
Generalização disjunta versus sobreposta:

- Generalização disjunta (ou exclusiva): Nesse caso, uma instância da superclasse só pode ser uma instância de uma (e apenas uma) subclasse. Por exemplo, em uma hierarquia Animal, uma instância específica pode ser um Gato ou um Cachorro, mas não ambos.
- Generalização sobreposta (ou não exclusiva): Nesse caso, uma instância da superclasse pode ser uma instância de mais de uma subclasse. Por exemplo, em uma hierarquia de Empregado, uma pessoa poderia ser simultaneamente uma instância de Programador e Designer.

11225354498 - Maria Vitória Gonçalves Vieira

Veja essas restrições anotadas nos diagramas abaixo:





Polimorfismo

Polimorfismo é o mecanismo da programação orientada a objetos que permite que objetos de diferentes classes sejam tratados como objetos de uma superclasse comum, enquanto ainda se comportam de acordo com suas características individuais.

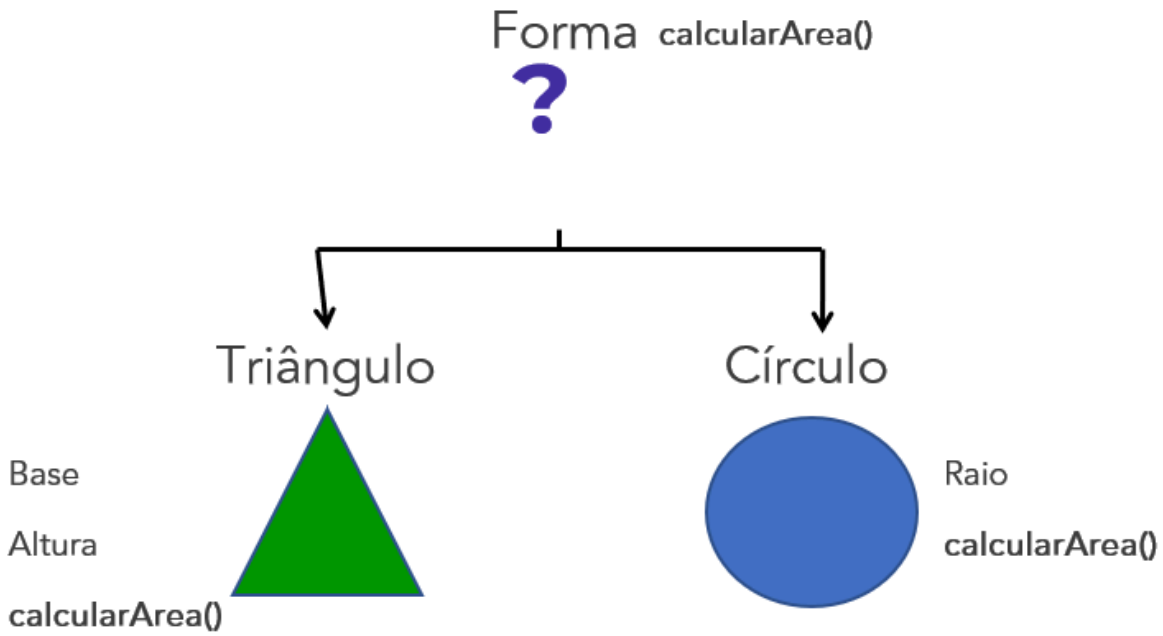
Em termos mais técnicos, o polimorfismo permite que referências de tipos de superclasse representem o comportamento dos tipos de subclasse. Isso significa que uma classe não precisa saber os detalhes de qualquer outra classe para interagir com ela, contanto que elas derivem da mesma superclasse.

Principais benefícios do polimorfismo:

- Reutilização de código: O polimorfismo permite reutilizar o código definindo uma interface comum para um grupo de classes. Por exemplo, você pode ter uma função que aceita um objeto de uma superclasse (como Forma), e essa função pode interagir com qualquer objeto de qualquer subclasse (Circulo, Triangulo, etc.) que herda da superclasse. Isso significa que você pode reutilizar a mesma função para muitos tipos diferentes de objetos.
- Flexibilidade e extensibilidade: As classes e funções que utilizam polimorfismo são mais flexíveis e extensíveis, porque podem lidar com novas subclasses sem serem modificadas. Se você adicionar uma nova subclasse, as funções existentes ainda poderão interagir com a nova classe, contanto que a nova classe siga a interface esperada.
- Abstração: O polimorfismo permite abstrair implementações específicas em uma interface comum. Isso significa que o código que usa essa interface não precisa saber detalhes sobre a implementação específica, tornando o código mais fácil de entender e manter.
- Substituição: Polimorfismo permite que um objeto seja substituído por um objeto de uma subclasse sem alterar a corretude do programa. Isso pode ser útil em muitos cenários, como quando você precisa substituir um objeto por uma versão mais especializada ou otimizada.
- Polimorfismo e Padrões de Projeto: O polimorfismo é a base de muitos padrões de projeto (design patterns) na programação orientada a objetos, como o padrão Strategy, onde um conjunto de algoritmos pode ser selecionado e trocado em tempo de execução.

Agora considere o exemplo das classes Forma, Triangulo e Circulo:

11225354498 - Maria Vitória Gonçalves Vieira



Expandindo o exemplo em código Python:

```python
class Forma:
    def area(self):
        pass

    def perimetro(self):
        pass

class Triangulo(Forma):
    def __init__(self, base, altura):
        self.base = base
        self.altura = altura

    def area(self):
        return 0.5 * self.base * self.altura

    def perimetro(self):
        # Para simplificar, vamos assumir que é um triângulo equilátero
        return 3 * self.base

class Circulo(Forma):
    def __init__(self, raio):
        self.raio = raio

    def area(self):
        return 3.14 * self.raio * self.raio

    def perimetro(self):
        return 2 * 3.14 * self.raio
```

Neste exemplo, Forma é a superclasse e Triangulo e Circulo são subclasses. Cada uma das subclasses define suas próprias implementações dos métodos area e perimetro.

Agora, podemos criar uma lista de formas, adicionar alguns triângulos e círculos e, em seguida, percorrer a lista e calcular a área e o perímetro de cada forma, independentemente de serem triângulos ou círculos. Essa "transparência" e comportamentos dinâmicos só são possíveis devido ao polimorfismo.

11225354498 - Maria Vitória Gonçalves Vieira

```
formas = [Triangulo(3, 4), Circulo(5), Triangulo(6, 8), Circulo(10)]
for forma in formas:
    print(f"Área: {forma.area()}, Perímetro: {forma.perimetro()}")
```

No código acima, cada chamada para forma.area() e forma.perimetro() invoca o método correto com base no tipo real do objeto. Isso é conhecido como ligação dinâmica ou ligação tardia, e é um dos principais recursos que tornam o polimorfismo possível.

O polimorfismo acima é conhecido como Polimorfismo Dinâmico, mas também há o polimorfismo Estático como vamos ver a seguir.

Polimorfismo Dinâmico (Overriding) versus Polimorfismo Estático (Overloading)

Polimorfismo Estático (Sobrecarga ou Overloading):

O polimorfismo estático, ou sobrecarga, ocorre quando duas ou mais funções na mesma escopo têm o mesmo nome mas parâmetros diferentes - seja um número diferente de parâmetros ou tipos de parâmetros diferentes. O compilador ou interpretador decide qual função usar com base nos argumentos passados na chamada da função. Esse processo é feito em tempo de compilação, daí o termo "estático".

Veja exemplo em uma linguagem que suporte sobrecarga de método, como Java ou C++:

```
void desenha(Forma forma) {
    // código para desenhar uma forma genérica
}

void desenha(Circulo circulo) {
    // código para desenhar um círculo
}

void desenha(Triangulo triangulo) {
    // código para desenhar um triângulo
}
```


11225354498 - Maria Vitória Gonçalves Vieira

Polimorfismo Dinâmico (Substituição ou Overriding):

O polimorfismo dinâmico, ou substituição, ocorre quando uma subclasse fornece uma implementação diferente de um método que já está fornecido por sua classe pai ou superclasse. O método na subclasse "sobrepõe" (ou "override") o método na classe pai. Nesse caso, o método a ser chamado é determinado em tempo de execução, com base no tipo real do objeto, daí o termo "dinâmico".

Já vimos um exemplo desse polimorfismo em Python. Veja o mesmo exemplo, dessa vez, em Java:

```java
class Forma {
    void desenha() {
        // código para desenhar uma forma genérica
    }
}

class Circulo extends Forma {
    @Override
    void desenha() {
        // código para desenhar um círculo
    }
}

class Triangulo extends Forma {
    @Override
    void desenha() {
        // código para desenhar um triângulo
    }
}
```

Em outras palavras, o polimorfismo estático (sobrecarga) ocorre em tempo de compilação e é baseado nos parâmetros da função. O polimorfismo dinâmico

(substituição), por outro lado, ocorre em tempo de execução e é baseado no tipo real do objeto.

Análise e Projeto OO

A Análise consiste em **atividades necessárias para entender o domínio do problema, isto é, o que deve ser feito**. É uma atividade de investigação, com foco no cliente. Já o Projeto consiste em atividades necessárias para entender o domínio da **solução** do problema, isto é, **como** deve ser feito. É uma atividade técnica, com foco no programador. Na Análise, a tecnologia de implementação e os requisitos não-funcionais não são modelados. Essa é uma tarefa do Projeto.

Na análise, também **não são pensadas soluções técnicas**, pensa-se apenas em modelar funções, dados e relacionamentos do sistema. O modelo de análise deve ser aprovado pelo cliente – pode incluir até pequenas discussões sobre a solução, como sobre Interfaces Gráficas. O Modelo de Casos de Uso representa o aspecto funcional de um domínio de negócio e, de forma similar, o Modelo de Classes representa o aspecto estrutural de um domínio de negócio.

O modelo de classes evolui à medida que o sistema é desenvolvido e é incrementado com novos detalhes. Existem três estágios sucessivos de abstração de modelo de classes: **análise, especificação e implementação**.

Modelo de Classes de Análise (ou Domínio)	Construído durante a atividade de análise, representa as classes de domínio do negócio. Não leva em consideração restrições inerentes à tecnologia a ser utilizada na solução de um problema.



11225354498 - Maria Vitória Gonçalves Vieira

Modelo de Classes de Especificação (ou Projeto)	Construído durante a atividade de projeto, estende o modelo de classes de análise e contém detalhes específicos inerentes à solução de software escolhida.



Modelo de Classes de Implementação	Construído durante a atividade de implementação, estende o modelo de classes de projeto e contém detalhes específicos inerentes ao desenvolvimento das classes em alguma linguagem.

```
1  private class Pessoa {
2      public String Nome;
3      public String Telefone;
4      public String Endereco;
5      private Empresa empresa[];
6
7      public String getInfo(){
8          return empresa.getConta();
9      }
10
11 public class Empresa {
12     public Number CNPJ;
13     public String Endereco;
14
15     public getConta() {
16         return 0;
17     }
18 }
```

À medida que o sistema é desenvolvido, o modelo é incrementado com novos detalhes. O Modelo de Análise enfatiza o desenho lógico, a visão externa, conceitual, abstrata, caixa-preta, de alto nível de abstração. O Modelo de Projeto enfatiza o desenho físico, a visão interna, de implementação, concreta, caixa-branca, de baixo nível de abstração. Sabe-se que o Modelo de Análise é mais estável que o Modelo de Projeto.

11225354498 - Maria Vitória Gonçalves Vieira

Sob a perspectiva organizacional, o primeiro sofre bem menos com mudanças tecnológicas, externas, regulatórias, etc. Além disso, é importante salientar que – na prática – muitas pessoas não fazem Análise e já partem para o Projeto. Inclusive, ela é uma disciplina opcional no RUP.

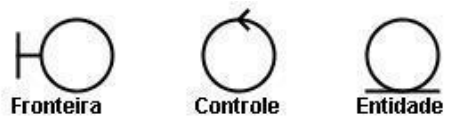
Com um modelo de análise pronto, torna-se muito mais fácil adaptar o modelo de projeto. Caso contrário, tem-se que refazer o modelo de projeto do início. Em geral, as classes de análise evoluem para classes de projeto. A meta da Análise é identificar um esboço preliminar do comportamento do sistema. A meta do Projeto é transformar esse esboço preliminar em um conjunto implementável.

O resultado é que há um refinamento detalhado e preciso quando alguém se move da Análise para o Design. Pessoal, a Análise apresenta quatro atividades principais:



Classes de Fronteira, Controle e Entidade

Em 1992, Ivar Jacobson propôs uma técnica chamada Análise de Robustez, que propunha a categorização das classes de acordo com sua responsabilidade: Classe de Fronteira; Classe de Controle; e Classe de Entidade.



CLASSE DE FRONTEIRA	Classe utilizada para modelar a interação entre um ator e o sistema. Para cada ator, é identificada pelo menos uma classe de fronteira para permitir sua interação com o sistema. Então, uma classe de fronteira existe para que o sistema se comunique com o mundo exterior, logo elas são altamente dependentes do ambiente.

A Interação entre sistema e atores envolve transformar e converter eventos, bem como observar mudanças na apresentação do sistema (como a interface). As classes de fronteira modelam as partes do sistema que dependem do ambiente. As classes de entidade e de controle modelam as partes que são independentes de fatores externos

ao sistema. Logo, alterar a GUI ou o protocolo de comunicação significa alterar só classes de fronteira e, não, classes de entidade e de controle.

As classes de fronteira também facilitam a compreensão do sistema, pois definem suas fronteiras. Elas ajudam no design, fornecendo um bom ponto de partida para identificar serviços relacionados. Algumas classes de fronteira comuns são janelas, protocolos de comunicação, interfaces de impressora, sensores e terminais. Se você estiver usando um construtor GUI, não será necessário modelar partes da interface de rotinas (botões, por exemplo) como classes de fronteira separadas

CLASSE DE controle	Classe utilizada para controlar a lógica de execução ou negócio correspondente a cada caso de uso. Servem como uma ponte de comunicação entre objetos de fronteira e objetos de entidade. Decidem o que o sistema deve fazer quando um evento externo relevante ocorre, agindo como coordenador para a realização de casos de uso.

Como objetos de controle (instâncias de classes de controle) geralmente controlam outros objetos, o comportamento de objetos de controle é do tipo coordenador. **As classes de controle encapsulam um comportamento específico de caso de uso**. O comportamento de um objeto de controle está estreitamente relacionado à realização de um caso de uso específico. Em muitos cenários, é possível até dizer que os objetos de controle "executam" as realizações de casos de uso.

No entanto, se as tarefas de caso de uso estiverem intrinsecamente relacionadas, alguns objetos de controle poderão participar de mais de uma realização de casos de uso. Além disso, vários objetos de controle de diferentes classes de controle podem participar de um único caso de uso. Nem todos os casos de uso exigem um objeto de controle. Se o fluxo de eventos em um caso de uso estiver relacionado a um objeto de entidade, um objeto de fronteira poderá realizar o caso de uso em cooperação com o objeto de entidade.

As classes de controle podem ajudar a entender o sistema, pois **representam a dinâmica do sistema, controlando as principais tarefas e os fluxos de controle**. Quando o sistema executar o caso de uso, um objeto de controle normalmente será criado. Os objetos de controle geralmente desaparecem após a execução do correspondente caso de uso. Observe que uma classe de controle não controla tudo o que é necessário em um caso de uso. Em vez disso, ela coordena as atividades de outros objetos que implementam a funcionalidade.

11225354498 - Maria Vitória Gonçalves Vieira

Classe de entidade	Classe utilizada para armazenar a informação que é manipulada ou processada pelo caso de uso, partindo do domínio do negócio. Geralmente, essas classes armazenam informações persistentes. Há várias instâncias ou objetos de uma mesma classe de entidade coexistindo dentro do sistema.

Os objetos de entidade (instâncias de classes de entidade) são usados para manter e atualizar informações sobre alguns fenômenos, como um evento, uma pessoa ou algum objeto real. Esses objetos geralmente são persistentes, precisando de atributos e relacionamentos durante muito tempo, algumas vezes durante todo o ciclo de vida do sistema. Um objeto de entidade geralmente não é específico para uma realização de casos de uso.

Os objetos de entidade representam os conceitos-chave do sistema que está sendo desenvolvido. Exemplos típicos de classes de entidade em um sistema bancário são Conta e Cliente. Em um sistema de gerenciamento de redes, os exemplos são Nó e Link. Se o fenômeno que você deseja modelar não for usado por outras classes, será possível modelá-lo como um atributo de uma classe de entidade ou mesmo como um relacionamento entre classes de entidade.

Projeto de Software

Uma vez que o sistema é modelado em alto nível na Análise, o foco muda para o Projeto de Software. O objetivo do projeto é determinar como o sistema deve funcionar para atender aos requisitos identificados na fase de análise. Essencialmente, é a fase em que as soluções técnicas para implementar os requisitos são definidas.

As atividades envolvidas no Projeto de Software incluem:

- Desenvolver uma arquitetura de software de alto nível para o sistema.
- Especificar estruturas de dados, interfaces e componentes.
- Decidir sobre tecnologias, frameworks e bibliotecas a serem usadas.
- Projetar algoritmos e determinar estratégias de implementação para os requisitos.
- Criar diagramas UML, como diagramas de classe, de sequência ou de estado.

O projeto arquitetural é, talvez, a atividade mais importante do Projeto de Software. Iremos tratar de arquitetura de software em uma aula separada, mas, em linhas gerais, a arquitetura representa a "macroestrutura" do sistema em termos de seus componentes mais significativos.

11225354498 - Maria Vitória Gonçalves Vieira

Uma maneira muito comum de organizar a arquitetura de software é utilizando camadas. A arquitetura em camadas permite melhor separação de responsabilidades; decomposição de complexidade; encapsulamento de implementação; maior reúso e extensibilidade. No entanto, podem penalizar o desempenho do sistema e aumentar o esforço/complexidade de desenvolvimento do software. Arquiteturas em duas camadas já foram dominantes, mas – para minimizar o impacto de mudanças – decidiu-se separar a camada de negócio da camada de interface gráfica, gerando três camadas:

Camada de apresentação	Possui classes que contêm funcionalidades para visualização dos dados pelos usuários (Por exemplo: classes de fronteiras para atores humanos). E qual a real importância dela? Ela tem o objetivo de exibir informações ao usuário e traduzir ações do usuário em requisições às demais partes dos sistemas.
Camada lógica de negócio	Possui classes que implementam as regras de negócio no qual o sistema será implantado. Ela realiza computações com base nos dados armazenados ou nos dados de entrada, decidindo que parte da camada de acesso deve ser ativada com base em requisições provenientes da camada de apresentação.
Camada de acesso	Possui classes que se comunicam com outros sistemas para realizar tarefas ou adquirir informações para o sistema. Tipicamente, essa camada é implementada utilizando algum mecanismo de armazenamento persistente. Pode haver uma subcamada dentro desta camada chamada Camada de Persistência.

Uma outra arquitetura muito famosa é o Modelo MVC, onde o sistema é dividido em três camadas da seguinte maneira:

Modelo	Responsável por modelar os dados da aplicação e regras de negócio. Tem o foco em armazenamento, manipulação e geração de dados. Objetos do Modelo são geralmente reusáveis, distribuídos, persistentes e portáveis.
Visão	Responsável pela apresentação dos dados aos usuários. Ele recebe entradas de dados e apresenta resultados. Essa camada não persiste nenhum dado no sistema e também não busca dados, apenas os renderiza em tela.
controle	Responsável por definir o comportamento da aplicação. Processa e responde a eventos, geralmente ações do usuário, e pode invocar alterações no modelo. Realiza, também, a validação de dados do usuário.

11225354498 - Maria Vitória Gonçalves Vieira

Por fim, nunca esqueça que a principal diferença entre Análise e Projeto reside no fato de que a Análise se concentra no "o quê" - o que o sistema deve fazer, enquanto o Projeto se concentra no "como" - como o sistema vai fazer isso. Ambas as atividades são essenciais para o desenvolvimento bem-sucedido de sistemas de software complexos.

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais[1].

Herança

Herança é um conceito fundamental na programação orientada a objetos (POO) que permite que uma classe herde estados (atributos) e comportamentos (métodos) de outra classe. Esse mecanismo cria uma hierarquia entre as classes, onde a classe que herda (conhecida como subclasse) pode reutilizar, estender ou modificar comportamentos da classe base (superclasse). A herança facilita a reutilização de código e a criação de relações "é um" entre objetos, simplificando o código e promovendo uma arquitetura mais limpa e organizada. Por exemplo, uma classe "Cachorro" pode herdar características de uma classe "Animal", indicando que todo "Cachorro" é um "Animal", e pode herdar métodos como "respirar" ou "comer", que são comuns a todos os animais.

Encapsulamento

Encapsulamento é outro pilar da programação orientada a objetos que trata da restrição do acesso direto a alguns dos componentes de um objeto. Esse princípio ajuda a proteger a integridade dos dados de um objeto, impedindo que sejam acessados ou modificados diretamente de fora da classe. Em vez disso, o acesso aos dados deve ser feito através de métodos públicos conhecidos como acessadores (getters) e modificadores (setters). Esse controle de acesso permite que a lógica interna de manipulação dos dados de um objeto seja separada de sua interface com o exterior. Isso não só fortalece a segurança dos dados, mas também torna o software mais fácil de testar e manter.

[1] Vale deixar claro que nem sempre será possível realizar uma aposta estratégica para um determinado assunto, considerando que às vezes não é viável identificar os pontos mais prováveis de serem cobrados a partir de critérios objetivos ou minimamente razoáveis.

11225354498 - Maria Vitória Gonçalves Vieira

Polimorfismo

Polimorfismo é a capacidade de um objeto ser referenciado de várias formas. Na prática, isso significa que um objeto de uma subclasse pode ser tratado como um objeto de sua superclasse, e o método específico que é executado pode ser determinado em tempo de execução. Isso é crucial para a flexibilidade e extensibilidade em sistemas de software, permitindo que novos tipos de objetos sejam adicionados ao sistema com alterações mínimas no código existente. Por exemplo, uma função que aceita um objeto da classe "Animal" pode operar usando um objeto "Cachorro" ou "Gato", e chamar métodos definidos na classe "Animal", mas que foram sobrescritos nas subclasses. O polimorfismo simplifica o gerenciamento de uma ampla gama de comportamentos relacionados e é uma ferramenta poderosa para a abstração em sistemas complexos.

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

1. (FGV / TCE-SE – 2015) Em POO (Programação Orientada a Objetos), dizer que a classe A estende a classe B é o mesmo que dizer que:

a) a classe B é subclasse de A;
b) a classe A é superclasse de B;
c) a classe A é derivada de B;
d) a classe B é derivada de A;
e) as classes A e B são irmãs.

Comentários: Se A estende B, A é derivada de B.

Gabarito: C

2. (FGV / TCE-SE – 2015) Em POO (programação orientada a objetos), dizer que a classe A é superclasse de B é o mesmo que dizer que:

a) A é derivada de B;
b) A estende B;
c) B é derivada de A;
d) B implementa A;
e) A implementa B.

Comentários: Essa ficou fácil! Se A é superclasse de B, B é derivada de A.

Gabarito: C

3. (FGV - 2022 - TCE-TO - Auditor de Controle Externo - Tecnologia da Informação) O sistema de controle automotivo SisAut foi modelado orientado a objetos. O SisAut possui a classe Veículo, que compartilha seu código-fonte com suas subclasses: Carro e Moto. As subclasses Carro e Moto aproveitam os métodos e atributos da classe Veículo. Em orientação a objeto, o mecanismo pelo qual uma classe pode estender outra classe ou ser estendida por outra classe é:

A) interface;
B) herança;
C) pacotes;
D) agregação;
E) encapsulamento.

Comentários: O mecanismo pelo qual uma classe pode estender outra classe ou ser estendida por outra classe é chamado de Herança – um dos princípios da orientação a objetos.

Gabarito: B

4. (FGV - 2016 - IBGE - Analista - Análise de Sistemas - Desenvolvimentos de Sistemas) Em Orientação a Objetos, para que uma subclasse de uma classe possa ter seu próprio comportamento, e mesmo assim compartilhar algumas das funcionalidades da classe pai, deve-se implementar:

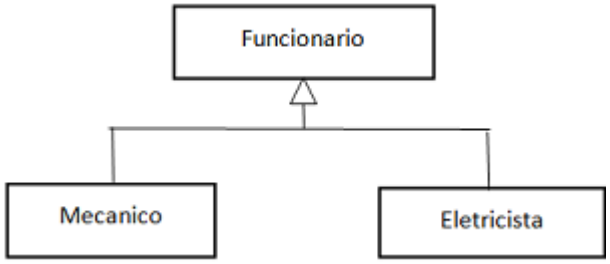
A) generalização;
B) agregação;
C) abstração;
D) composição;
E) polimorfismo.

Comentários: Quando uma subclasse tem seu comportamento especializado, apesar de compartilhar funcionalidades com a superclasse estendendo o comportamento original, temos um caso de polimorfismo.

Gabarito: E

11225354498 - Maria Vitória Gonçalves Vieira

5. (FGV - 2016 - IBGE - Analista - Análise de Sistemas - Desenvolvimento de Aplicações - Web Mobile) A figura abaixo representa o diagrama de classes de parte do sistema SOMANUT. Na classe Funcionario, foi implementado o método abstrato calcularSalario.



Para que a classe Mecanico possa ser instanciada, é necessário que a(s) classe(s):

A) Funcionario tenha definido um atributo salario que seja protegido;
B) Mecanico redefina o método calcularSalario;
C) Funcionario tenha definido um atributo salario que seja público;
D) Mecanico e Eletricista redefinam o método calcularSalario;
E) Funcionario possua outros métodos concretos.

Comentários: A classe Mecanico tem que redefinir o método calcularSalario como um método concreto, do contrário, ela própria teria que ser uma classe abstrata.

Gabarito: B

QUESTIONÁRIO DE REVISÃO E APERFEIÇOAMENTO

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma auto explicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

11225354498 - Maria Vitória Gonçalves Vieira

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível!

Vamos ao nosso questionário:

Perguntas

1. O que é o paradigma orientado a objetos?
2. Como a programação orientada a objetos se compara à programação estruturada?
3. O que é um objeto em programação orientada a objetos?
4. O que é uma classe?
5. O que é uma classe abstrata?
6. O que são atributos de classe e atributos de objeto?
7. O que é um método construtor?
8. O que é o conceito de Mensagens em POO?
9. O que é abstração?
10. O que é uma Interface?
11. Quais são os benefícios do encapsulamento?
12. O que é herança?
13. Por que a herança múltipla geralmente não é recomendada?
14. O que é polimorfismo?
15. Qual é a diferença entre polimorfismo estático (overloading) e polimorfismo dinâmico (overriding)?
16. Qual é o objetivo de Projeto de Software e como ele se diferencia da Análise?
17. O que é Early Binding e Late Binding?
18. O que são Modificadores de Acesso?
19. O que é uma Classe de Fronteira?

20. O que é uma Classe de Controle?

Perguntas e Respostas

1. O que é o paradigma orientado a objetos?
   Resposta: O paradigma orientado a objetos é um estilo de programação que envolve a organização de códigos em torno de "objetos" - entidades que contêm dados (atributos) e comportamentos (métodos). Este paradigma facilita a abstração, a herança, o encapsulamento e o polimorfismo.
2. Como a programação orientada a objetos se compara à programação estruturada?
   Resposta: A programação orientada a objetos permite uma abordagem mais modular e escalável para o desenvolvimento de software, onde os objetos podem ser reutilizados e interagem uns com os outros. Em contraste, a programação estruturada se concentra no fluxo sequencial de controle, tornando-a menos flexível para projetos de software grandes e complexos.
3. O que é um objeto em programação orientada a objetos?
   Resposta: Um objeto é uma instância de uma classe. Ele possui um estado, que é determinado pelos valores de seus atributos, e comportamentos, que são definidos por seus métodos.
4. O que é uma classe?
   Resposta: Uma classe é um plano ou modelo que define as características (atributos) e ações (métodos) que um objeto pode ter. A classe serve como um molde para criar objetos.
5. O que é uma classe abstrata?
   Resposta: Uma classe abstrata é uma classe que não pode ser instanciada e é geralmente usada como uma classe base para outras classes. Ela pode conter métodos abstratos (métodos sem implementação) que devem ser implementados por quaisquer subclasses concretas.
6. O que são atributos de classe e atributos de objeto?
   Resposta: Atributos de classe são variáveis que são compartilhadas por todas as instâncias de uma classe, enquanto atributos de objeto são variáveis que pertencem a uma instância específica de uma classe.
7. O que é um método construtor?
   Resposta: Um método construtor é um método especial de uma classe usado para inicializar novos objetos dessa classe.
8. O que é o conceito de Mensagens em POO?
   Resposta: Em POO, uma mensagem é uma chamada a um objeto para invocar um de seus métodos. Basicamente, é a forma de comunicação entre objetos.

11225354498 - Maria Vitória Gonçalves Vieira

9. O que é abstração?
   Resposta: Abstração é um conceito que permite modelar características complexas por meio de características mais simples e relevantes. Em outras palavras, a abstração permite focar no que um objeto faz em vez de como ele faz.
10. O que é uma Interface?
    Resposta: Uma interface é um contrato dentro de um sistema orientado a objetos que especifica quais métodos uma classe deve implementar. As interfaces são usadas para definir comportamentos específicos.
11. Quais são os benefícios do encapsulamento?
    Resposta: O encapsulamento protege os dados e impede que sejam acessados diretamente. Ele aumenta a segurança, flexibilidade e facilidade de manutenção do código.
12. O que é herança?
    Resposta: Herança é um mecanismo pelo qual uma classe pode herdar características (atributos e métodos) de outra classe. A classe de onde as características são herdadas é chamada de superclasse ou classe pai, e a classe que herda as características é chamada de subclasse ou classe filha.
13. Por que a herança múltipla geralmente não é recomendada?
    Resposta: A herança múltipla pode levar a uma situação chamada "problema do diamante" onde a hierarquia de herança pode levar a ambiguidades e confusão. É por isso que muitas linguagens de programação (como Java e C#) não suportam herança múltipla.
14. O que é polimorfismo?
    Resposta: Polimorfismo é a capacidade de um objeto ser interpretado como mais de um tipo. Ele permite que o mesmo nome de método seja usado para métodos que realizam tarefas ligeiramente diferentes, dependendo do tipo do objeto que o método está sendo chamado.
15. Qual é a diferença entre polimorfismo estático (overloading) e polimorfismo dinâmico (overriding)?
    Resposta: Overloading (polimorfismo estático) é quando dois ou mais métodos na mesma classe têm o mesmo nome, mas diferentes parâmetros. Overriding (polimorfismo dinâmico) é quando uma subclasse fornece uma implementação diferente de um método já existente na superclasse.
16. Qual é o objetivo de Projeto de Software e como ele se diferencia da Análise?
    Resposta: O objetivo do Projeto de Software é determinar como o sistema deve funcionar para atender aos requisitos identificados na fase de análise. A Análise se concentra no "o quê" - o que o sistema deve fazer, enquanto o Projeto se concentra no "como" - como o sistema vai fazer isso.
17. O que é Early Binding e Late Binding?
    Resposta: Early Binding (Ligação Precoce) é quando o método a ser executado é

11225354498 - Maria Vitória Gonçalves Vieira

determinado em tempo de compilação. Late Binding (Ligação Tardia) é quando o método a ser executado é determinado em tempo de execução.

18. O que são Modificadores de Acesso?
    Resposta: Modificadores de acesso são palavras-chave que definem a visibilidade dos atributos, métodos e outras partes de uma classe ou estrutura.
19. O que é uma Classe de Fronteira?
    Resposta: Uma Classe de Fronteira é uma classe que controla a interação entre o sistema e o ambiente externo, geralmente através de interfaces de usuário.
20. O que é uma Classe de Controle?
    Resposta: Uma Classe de Controle é uma classe responsável por gerenciar o fluxo de controle entre classes de fronteira e classes de entidade.

Lista de Questões Estratégicas

1. (FGV / TCE-SE – 2015) Em POO (Programação Orientada a Objetos), dizer que a classe A estende a classe B é o mesmo que dizer que:

a) a classe B é subclasse de A;
b) a classe A é superclasse de B;
c) a classe A é derivada de B;
d) a classe B é derivada de A;
e) as classes A e B são irmãs.

2. (FGV / TCE-SE – 2015) Em POO (programação orientada a objetos), dizer que a classe A é superclasse de B é o mesmo que dizer que:

a) A é derivada de B;
b) A estende B;
c) B é derivada de A;
d) B implementa A;
e) A implementa B.

3. (FGV - 2022 - TCE-TO - Auditor de Controle Externo - Tecnologia da Informação) O sistema de controle automotivo SisAut foi modelado orientado a objetos. O SisAut possui a classe Veículo, que compartilha seu código-fonte com suas subclasses: Carro e Moto. As subclasses Carro e Moto aproveitam os métodos

11225354498 - Maria Vitória Gonçalves Vieira

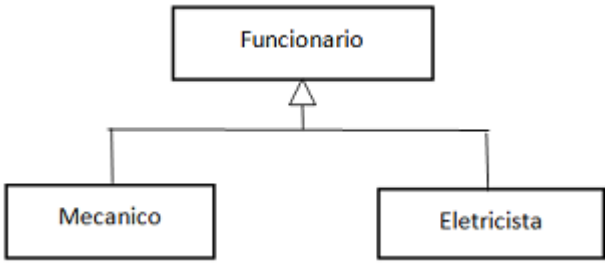
e atributos da classe Veículo. Em orientação a objeto, o mecanismo pelo qual uma classe pode estender outra classe ou ser estendida por outra classe é:

A) interface;
B) herança;
C) pacotes;
D) agregação;
E) encapsulamento.

4. (FGV - 2016 - IBGE - Analista - Análise de Sistemas - Desenvolvimentos de Sistemas) Em Orientação a Objetos, para que uma subclasse de uma classe possa ter seu próprio comportamento, e mesmo assim compartilhar algumas das funcionalidades da classe pai, deve-se implementar:

A) generalização;
B) agregação;
C) abstração;
D) composição;
E) polimorfismo.

5. (FGV - 2016 - IBGE - Analista - Análise de Sistemas - Desenvolvimento de Aplicações - Web Mobile) A figura abaixo representa o diagrama de classes de parte do sistema SOMANUT. Na classe Funcionario, foi implementado o método abstrato calcularSalario.



Para que a classe Mecanico possa ser instanciada, é necessário que a(s) classe(s):

A) Funcionario tenha definido um atributo salario que seja protegido;
B) Mecanico redefina o método calcularSalario;
C) Funcionario tenha definido um atributo salario que seja público;
D) Mecanico e Eletricista redefinam o método calcularSalario;
E) Funcionario possua outros métodos concretos.

11225354498 - Maria Vitória Gonçalves Vieira

Gabaritos

1. C
2. C
3. B
4. E
5. B

11225354498 - Maria Vitória Gonçalves Vieira

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.